

Pontificia Universidad Católica Madre y Maestra
(PUCMM)



“Práctica #1 - Cliente HTTP”

Estudiante:
Samuel Olivo Pichardo
10154465

Asignatura:
Programación Web
CSTI-1910-6339

Docente:
Carlos Alfredo Camacho Guerrero

Santiago de los Caballeros, República Dominicana
9 de febrero de 2026

Introducción

En este proyecto se desarrolla un cliente HTTP en Java utilizando la librería HttpClient. El programa permite al usuario ingresar una URL, validar su acceso y determinar el tipo de recurso solicitado. En caso de tratarse de un documento HTML, el cliente analiza su contenido, contando líneas, párrafos, imágenes y formularios, mostrando además los campos de entrada y enviando datos a los formularios que utilicen el método POST. Este proyecto me permitió comprender de manera práctica cómo funcionan las peticiones HTTP y el procesamiento de documentos HTML en Java.

Desarrollo

```
plugins {
    id 'java'
    id 'application'
}

application {
    mainClass = 'org.example.Main'
}

group = 'org.example'
version = '1.0-SNAPSHOT'

repositories {
    mavenCentral()
}

dependencies {
    implementation 'io.javalin:javalin:6.7.0'
    implementation("org.slf4j:slf4j-simple:2.0.10")

    implementation 'org.jsoup:jsoup:1.22.1'

    testImplementation platform('org.junit:junit-bom:5.10.0')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

test {
    useJUnitPlatform()
}
```

```
package org.example;

import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.Scanner;

import static java.lang.System.in;
import static java.lang.System.out;

public class Main { @ Samuel Olivo
    static void main() throws IOException, InterruptedException { @ Samuel Olivo
        cliente();
    }
}
```

Estas fueron las dependencias y librerías utilizadas. La función cliente() contiene toda la lógica necesaria. Para el funcionamiento adecuado de la práctica.

```
static void cliente() throws IOException, InterruptedException { 2 usages @ Samuel Olivo
    Scanner scanner = new Scanner(in);
    String input;
    HttpRequest request;

    //Para capturar URL
    do {
        out.print("Ingrese URL válida: ");
        input = scanner.nextLine();
        request = requestValido(input);
    } while (request == null);

    //Para determinar el tipo de recurso
    HttpResponse<String> response = HttpClient.newHttpClient()
        .send(request, HttpResponse.BodyHandlers.ofString());

    String tipoRecurso = response.headers().get(HttpHeaders
        .contentType)
        .orElse("desconocido");

    out.println("Tipo de recurso seleccionado: " + tipoRecurso);

    //Informacion adicional en caso de ser html
    if (tipoRecurso.contains("html")){
        htmlInfo(response);
    }
}
```

Se valida la URL proporcionada, y se examina el tipo del recurso. En caso de ser HTML, se procede con el análisis. Aquí presento la lógica principal de extraer los formularios y responder con un post al servidor de donde proviene el recurso.

```
Elements inputs;
int cont = 0;
for (Element f : forms) {
    cont++;

    System.out.println("\n\nForm " + cont + ": ");
    f.attributes().forEach( Attribute attr -> System.out.print(attr.getKey() + "=" + attr.getValue() + "    ")

    inputs = f.select( cssQuery: "input");
    System.out.println("\nInputs: ");
    for (Element i : inputs){
        i.attributes().forEach( Attribute attr -> System.out.println(attr.getKey() + "=" + attr.getValue()));
    }

    if (f.attr( attributeKey: "method").equalsIgnoreCase( anotherString: "post")){

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(f.absUrl( attributeKey: "action")))
            .header( name: "matricula-id", value: "10154465")
            .header( name: "Content-Type", value: "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString( body: "asignatura=practica1"))
            .build();

        HttpClient client = HttpClient.newHttpClient();
        HttpResponse<String> responsePOST = client.send(request, HttpResponse.BodyHandlers.ofString());

        out.println("\nRespuesta: ");
        out.println(responsePOST.toString());
    }
}
```

Decidí implementar recursividad en la función cliente() para permitir el ingreso de múltiples URL. Lo anterior y la implementación lambda en:

```
f.attributes().forEach(attr -> System.out.print(attr.getKey() + "=" + attr.getValue() + "    "));
```

Son un producto de poner a prueba los aprendizajes adquiridos en la materia de programación funcional.

```
char tecla;
do {
    out.println("\n\n¿Quieres probar con otra URL? [S]i [N]o: ");
    tecla = scanner.next().charAt(0);
    if (tecla == 's' || tecla == 'S')
        cliente();
} while(!(tecla == 's' || tecla == 'n' || tecla == 'S' || tecla == 'N'));
System.exit( status: 0);
```

Conclusión

Este proyecto muestra cómo se puede interactuar con recursos web, extrayendo información útil de documentos HTML y realizando envíos de datos a formularios mediante HTTP POST. Una aplicación práctica de este conocimiento podría ser la automatización de pruebas de formularios en páginas web educativas, permitiendo verificar el correcto funcionamiento de los mismos o recolectar datos estructurados para análisis sin intervención manual. Además, se fortalece la comprensión del protocolo HTTP y del manejo de datos en Java.

Bibliografia:

Hedley, J. (s. f.). *jsoup: Java HTML parser, built for HTML editing, cleaning, scraping, and XSS safety*. Jsoup: Java HTML Parser. <https://jsoup.org/>

Introduction to the Java HTTP Client. (s. f.). <https://openjdk.org/groups/net/httpclient/intro.html>

Repositorio:

<https://github.com/samuelolivo/st-icc-352-002-sxop0001/tree/main/practica-1>